PROGRAMACIÓN ASISTIDA DE APLICACIONES · USFQ · SEMANA 7

DIGEST · divisor_cuenta

Todo lo que necesitas saber del código del divisor de cuenta: cómo está armado, por qué, y cómo explicarlo. Primero la versión profunda; después, la misma historia contada para alguien de 8 años.

6/6 escenarios en la rama sdd · 33/33 pruebas en verde · flutter analyze: sin issues · 11 archivos · 523 líneas en lib/ · pendiente: APK y GitHub

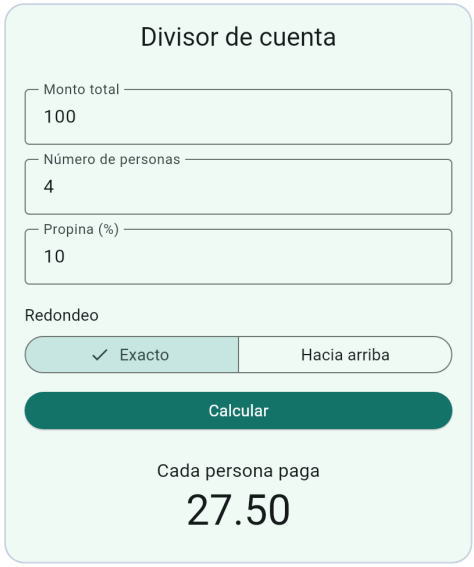
Cómo leer este documento

1. **Parte A, versión profunda** (págs. 2-10): arquitectura, decisiones, flujo de datos, archivos clave, SOLID, pruebas, seguridad, límites y cómo extenderlo.
2. **Parte B, versión para niños** (págs. 11-13): los mismos conceptos con personajes, un cómic y un pastel.
3. **Glosario** y **dónde mirar primero en el código** (págs. 14-15).

En 30 segundos

Escribes **monto**, **personas** y **propina**, eliges **Exacto** o **Hacia arriba** y tocas **Calcular**. La app valida, calcula `monto × (100 + propina) / (100 × personas)`, redondea con la estrategia elegida y muestra el resultado con dos decimales.

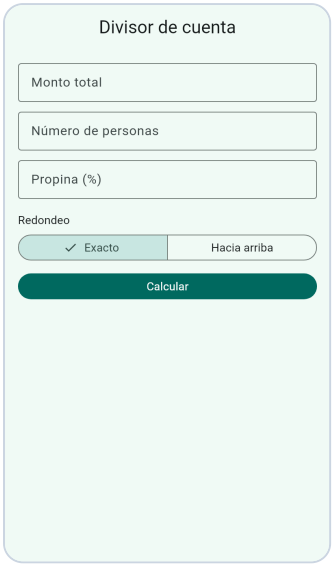
El código vive en tres capas: **presentation** (Flutter), **domain** (Dart puro, las reglas) y **data** (los redondeos). `main.dart` es el único lugar donde se crean las piezas concretas.



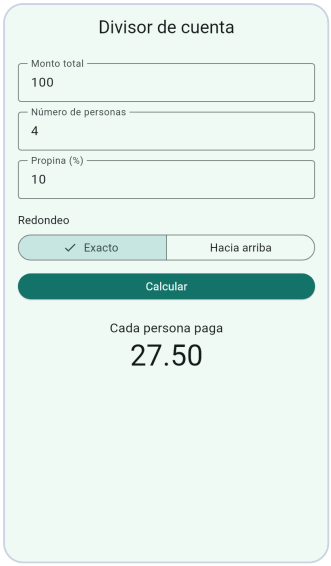
100 · 4 · 10 % → 27.50

Las capturas son de la app real, corriendo en Chrome el 2026-10-01. Rama de referencia: `sdd`, commit `7ffc049`.

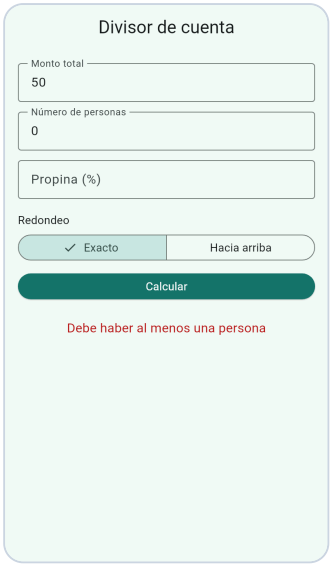
A1 · Qué hace la app



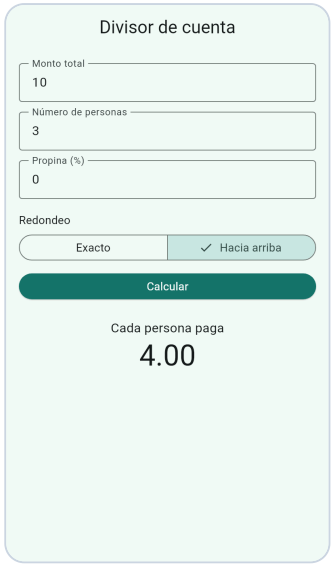
Al abrir: modo Exacto elegido



Escenario 1: 27.50

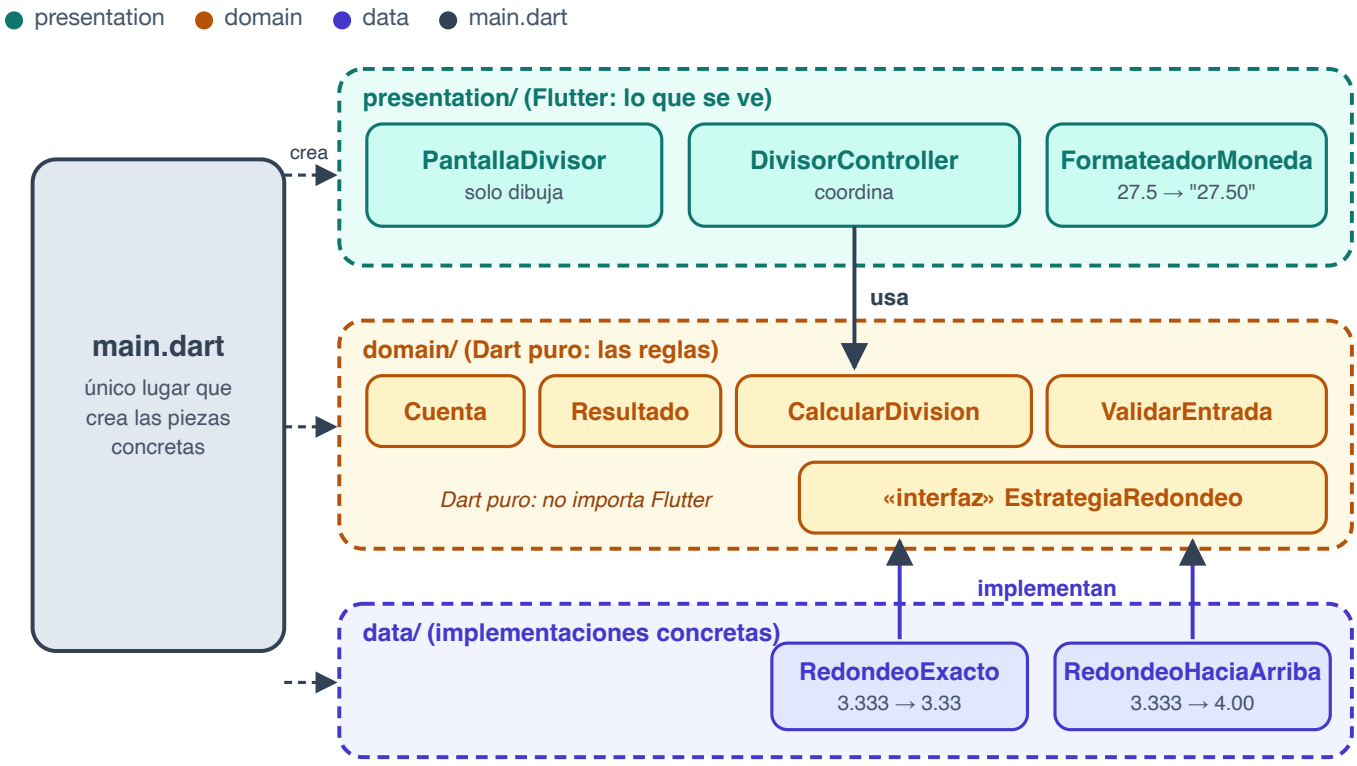


Escenario 3: 0 personas



Escenario 6: hacia arriba → 4.00

A2 · Arquitectura: tres capas y un armador



La regla de dependencia es `presentation → domain ← data`. Las flechas apuntan hacia el domain: él no conoce a nadie. Por eso `lib/domain/` no importa Flutter y se puede probar sin abrir ninguna pantalla. `data` también depende del domain: implementa la interfaz `EstrategiaRedondeo` que el domain define. Esto es **inversión de dependencias**: la regla importante (el domain) define el contrato y el detalle (un redondeo concreto) lo cumple.

¿Por qué `main.dart` está aparte? Alguien tiene que crear las piezas concretas y conectarlas. Si ese "alguien" fuera la pantalla, la pantalla dependería de `data` y la regla se rompería. Concentrarlo en un solo archivo (la *raíz de composición*) hace que cambiar o agregar una pieza sea editar un solo lugar.

A3 · Mapa de carpetas

Las barras muestran el tamaño de cada archivo. Fíjate en que todo el domain junto (127 líneas) es más chico que la pantalla sola: la lógica es poca y está concentrada; lo que ocupa espacio es dibujar.

Archivo	Para qué existe	Líneas
`lib/main.dart`	arma todo: crearControlador() + runApp	36
`lib/domain/cuenta.dart`	entidad: monto, personas, propina	26
`lib/domain/resultado.dart`	entidad: montoPorPersona	11
`lib/domain/estrategia_redondeo.dart`	interfaz de 1 método: redondear()	12
`lib/domain/calcular_division.dart`	caso de uso: la fórmula	27
`lib/domain/validar_entrada.dart`	caso de uso: reglas y mensajes	51
`lib/data/redondeo_exacto.dart`	al centavo más cercano	21
`lib/data/redondeo_hacia_arriba.dart`	al entero siguiente	25
`lib/presentation/formateador_moneda.dart`	número → "27.50"	14
`lib/presentation/divisor_controller.dart`	texto → validar → calcular → texto	135
`lib/presentation/pantalla_divisor.dart`	widgets y setState	165

Fuera de lib/

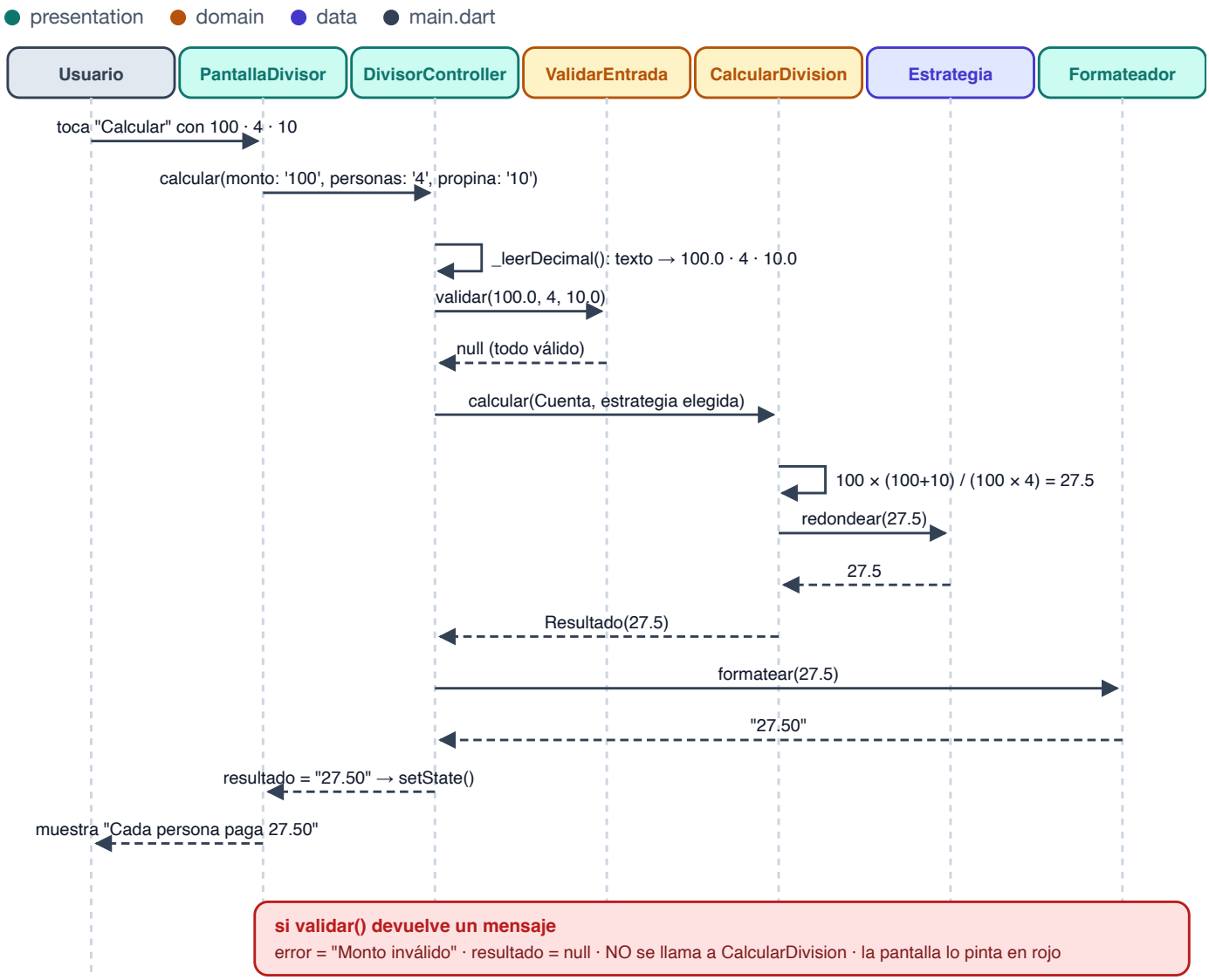
Ruta	Qué es
`test/`	5 archivos, 33 pruebas (ver A9)
`specs/001-divisor-cuenta/`	spec, plan, research, data-model, contracts, quickstart, tasks
`.specify/memory/constitution.md`	las reglas permanentes (SOLID, capas…)
`.specify/` , `.claude/skills/`	Spec Kit: plantillas, scripts y los `/speckit-*`
`CLAUDE.md`	instrucciones para el agente
`analysis_options.yaml`	lints, con `public_member_api_docs`
`android/ ios/ macos/ web/ …`	plantilla de Flutter, sin tocar

Qué importa a qué

```
domain/   → solo domain/
data/     → domain/ + dart:math
presentation/ → domain/ + flutter
main.dart → todo (es el armador)

grep -rn "package:flutter" lib/domain/
  (sin salida)
grep -rn "data/" lib/presentation/
  (sin salida)
```

A4 · Flujo de datos: qué pasa cuando tocas "Calcular"



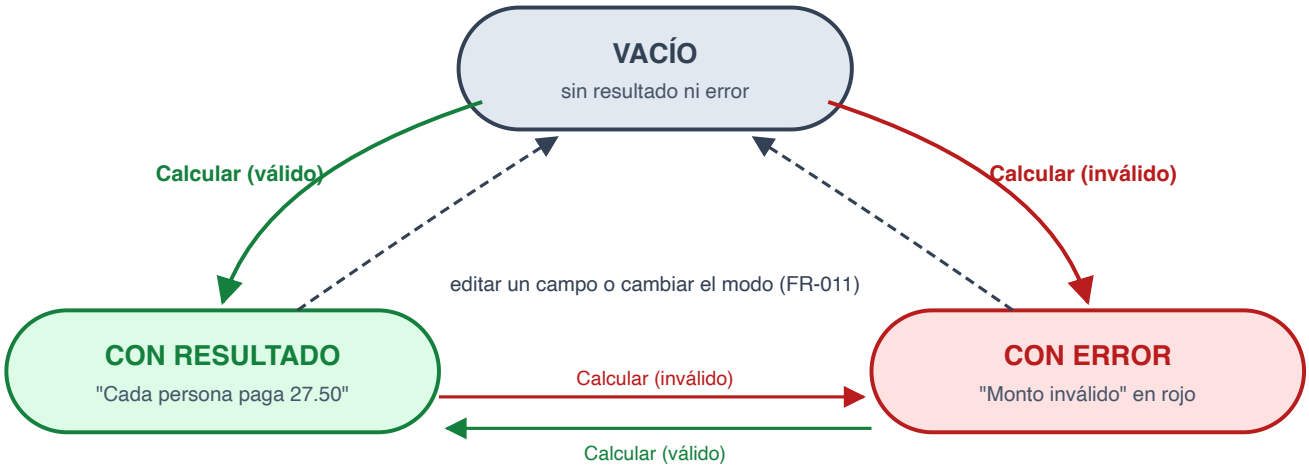
> **Dos detalles que valen oro en la defensa oral.** (1) El texto se convierte a número en el **controlador**, pero la decisión de si ese número es válido la toma **ValidarEntrada**: así los mensajes viven en un solo archivo. (2) La fórmula multiplica antes de dividir: `100 × 110 / 400` da 27.5 exacto, mientras que `100 × 1.1 / 4` da 27.500000000000004 porque 1.1 no se puede guardar exacto en binario.

El corazón del programa, entero

```
Resultado calcular(Cuenta cuenta, EstrategiaRedondeo estrategia) {
  final montoSinRedondear =
      cuenta.monto * (100 + cuenta.propina) / (100 * cuenta.personas);
  return Resultado(estrategia.redondear(montoSinRedondear));
}
```

Fíjate en lo que **no** tiene: ni `if` sobre el tipo de estrategia, ni mensajes de error, ni `toStringAsFixed`. Esas ausencias son SRP, OCP y LSP a la vez.

A5 · Flujo del usuario: los tres estados de la pantalla



Nunca hay resultado y error a la vez. Editar cualquier campo o cambiar el modo vuelve a **VACÍO** para que nunca quede en pantalla un número que no corresponde a lo escrito (esto salió de `/speckit-clarify`).

A6 · Decisiones técnicas y su porqué

Decisión	Por qué	Alternativa descartada
`setState`, sin Provider/BLoC	Es una pantalla; el plan lo pidió así y no se permiten paquetes externos.	Provider (paquete externo), BLoC (sobra)
`double` + tolerancia	Los casos de prueba de la guía usan `double` (incluso `double.nan`). Con los límites de entrada, los centavos caben en la precisión.	centavos como `int`
Exacto = centavo más cercano, mitad arriba	La prueba compara 10/3 con 3.33 ± 0.001: hay que redondear en el domain, no solo al mostrar.	redondear solo al formatear
Hacia arriba con tolerancia relativa	`30.000000000000004` no debe subir a 31, pero `12.0033` sí debe subir a 13 para cubrir la cuenta.	redondear a centavos y luego subir
`validar()` devuelve `String?`	null = válido; si no, el mensaje exacto. Fácil de explicar y de probar.	clase sellada Válido/Inválido
Modos como `Map<String, EstrategiaRedondeo>`	La pantalla dibuja un botón por entrada: un modo nuevo = una clase nueva + su entrada en el mapa de `main.dart` (OCP).	`enum` + `switch`
Personas: solo dígitos, máx. 6	Elimina "2.5", "dos" y "-3" desde el teclado y evita desbordar `int`.	validar texto libre
Límites: monto ≤ 1e9, propina ≤ 100 %	Garantizan centavos correctos; una propina mayor que la cuenta es casi seguro un error.	sin límites
Lint `public_member_api_docs`	Hace que `flutter analyze` exija la documentación que pide la regla de la materia.	confiar en la buena voluntad
Estrategias sin constructor propio	Así `grep "RedondeoExacto()"` solo encuentra `main.dart` (lo detectó converge, T028).	`const RedondeoExacto();`

A7 · Archivos y funciones clave

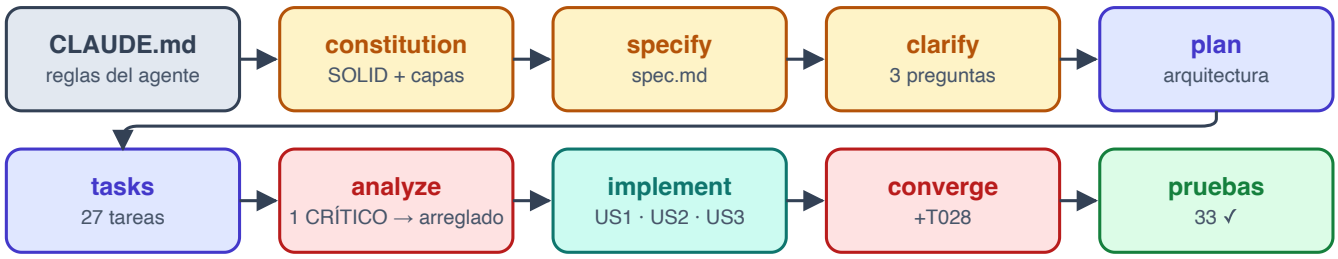
La regla V de la constitución dice que debes poder explicar cada función: qué hace, por qué existe, qué recibe, qué devuelve y qué errores produce. Esta tabla es tu chuleta; el mismo texto está en los comentarios `///` del código.

Función	Qué hace / por qué existe	Recibe	Devuelve	Errores
`CalcularDivision.calcular`	Aplica la fórmula y delega el redondeo. Existe para que la fórmula viva en un solo lugar.	`Cuenta` ya validada, `EstrategiaRedondeo`	`Resultado`	ninguno si la cuenta es válida
`ValidarEntrada.validar`	Revisa monto → personas → propina y devuelve el primer error. Todas las reglas y mensajes juntos.	`double` monto (NaN si no es número), `int` personas, `double` propina	`null` o el mensaje	no lanza: un dato malo es un resultado esperado
`EstrategiaRedondeo.redondear`	Contrato de un método para cualquier regla de redondeo (ISP, OCP).	monto por persona	monto redondeado	—
`RedondeoExacto.redondear`	Al centavo, mitad hacia arriba, con margen `1e-6` contra el ruido binario.	3.3333	3.33	ninguno
`RedondeoHaciaArriba.redondear`	Al entero siguiente salvo que ya sea entero.	3.3333 / 30.0	4.0 / 30.0	ninguno
`FormateadorMoneda.formatear`	Único lugar que decide cómo se ve un número.	27.5	`"27.50"`	ninguno
`DivisorController(...)`	Recibe sus piezas por constructor (DIP). El primer modo del mapa es el inicial.	validar, calcular, formateador, mapa de estrategias	—	`ArgumentError` si el mapa está vacío
`DivisorController.calcular`	Convierte texto a número, valida y, solo si es válido, calcula y formatea.	3 `String`	nada: actualiza `resultado` / `error`	ninguno
`DivisorController.seleccionarModo`	Cambia el modo y limpia la pantalla.	nombre del modo	nada	`ArgumentError` si no existe
`DivisorController.limpiar`	Borra resultado y error al editar (FR-011).	nada	nada	ninguno
`crearControlador()` (main)	Raíz de composición: crea todo lo concreto.	nada	`DivisorController`	ninguno
`PantallaDivisor`	Dibuja 3 campos, el selector, el botón y el resultado o el error. Llama al controlador dentro de `setState`.	el controlador	widgets	—

A8 · SOLID en este código, con la prueba de cada uno

Principio	Dónde se ve	Cómo se comprueba
SRP una razón de cambio	`CalcularDivision` solo calcula, `ValidarEntrada` solo valida, `FormateadorMoneda` solo formatea, `PantallaDivisor` solo dibuja.	`grep -n -E "toStringAsFixed|inválido|al menos una persona" lib/domain/calcular_division.dart` → sin salida
OCP abierto a extender	"Hacia arriba" se agregó con un archivo nuevo y 5 líneas en `main.dart` (commit `bb1ae56`); ninguna clase existente cambió.	`git show --stat bb1ae56` → solo `main.dart` y el archivo nuevo
LSP sustituibles	`calcular()` funciona igual con cualquier estrategia, sin preguntar su tipo.	`grep -n -E "is Redondeo|as Redondeo" lib/domain/calcular_division.dart` → sin salida; prueba "LSP" en `division_test.dart`
ISP interfaces chicas	`EstrategiaRedondeo` tiene un único método.	leer `estrategia_redondeo.dart` (12 líneas)
DIP depender de abstracciones	El controlador recibe todo por constructor y solo conoce la interfaz de redondeo.	`grep -rn "data/" lib/presentation/` → sin salida; `grep -rn -E "RedondeoExacto\(\)|RedondeoHaciaArriba\(\)" lib/` → solo `main.dart`

Cómo llegó el código hasta aquí: el flujo SDD



Lo que el flujo atrapó antes que un humano: `/speckit-analyze` vio que el `widget_test.dart` de la plantilla iba a romper `flutter analyze` apenas cambiara `main.dart`; se arregló en `tasks.md` antes de escribir código. `/speckit-converge` vio que el `grep` de la constitución también encontraba el constructor `const RedondeoExacto();` en `lib/data/`; se convirtió en la tarea T028. En los dos casos se corrigió el **artefacto**, no se parchó el código a ciegas.

A9 · Pruebas: 33, en cinco archivos

Archivo	Cuántas	Qué cubren	¿Usa widgets?
`casos_de_prueba.dart`	(datos)	los 6 escenarios de la spec, copiados tal cual de la guía	no
`division_test.dart`	7	un test por escenario + uno de LSP; un "espía" cuenta que con error **no se calcula**	no
`pantalla_test.dart`	3	27.50 · cero personas sin resultado · "abc" → "Monto inválido"	sí
`casos_borde_test.dart`	19	límites, ruido binario (1.005 → 1.01), 999 999 personas, coma decimal, orden de errores, modos	no
`pantalla_borde_test.dart`	4	modo "Hacia arriba" en la UI, borrar al editar, campo de personas, nunca resultado + error	sí

> **¿Las pruebas sirven de verdad?** Se rompió el redondeo a propósito (devolver el número sin redondear) y fallaron el escenario 5 y la prueba de LSP. Se restauró y volvió a 33/33. Una prueba que nunca puede fallar no prueba nada.

Por qué dos archivos para los escenarios. `casos_de_prueba.dart` es la traducción literal de la spec (se lee sin saber Flutter); `division_test.dart` los recorre. Si mañana agregas un escenario, tocas solo el primero. Eso es SRP aplicado a las pruebas.

A10 · Seguridad

Tema	Estado
Secretos y API keys	No hay. `grep -rniE "api[_-]?key|secret|password|token" lib/` no devuelve nada; `.env*` está en `.gitignore`.
Red y datos personales	La app no usa red ni guarda nada: no hay qué filtrar ni qué robar.
Validación de entrada	Todo número pasa por `ValidarEntrada`: NaN, infinito, negativos, cero y valores enormes se rechazan con un mensaje.
Dependencias	Solo el SDK de Flutter y `flutter_lints` (de desarrollo). Se quitó `cupertino_icons`, que no se usaba. `flutter pub get` no reportó avisos de seguridad.
Superficie de ataque	Mínima: una pantalla local. El riesgo real es un resultado incorrecto, y para eso están las 33 pruebas.

A11 · Límites conocidos

Límite	Detalle
APK sin construir	Esta Mac no tiene Android SDK; `flutter build macos --debug` sí construye. Pasos en `MY_STEPS.md`.
Centavo que sobra	10/3 = 3.33 por persona y la suma da 9.99. Es intencional (clarify, FR-013), pero el grupo queda corto 1 centavo.
"abc" en un teléfono	El campo del monto abre un teclado numérico; para escribir letras hay que pegarlas o usar un teclado físico.
Formato fijo	Siempre punto decimal y sin símbolo de moneda; no se adapta al idioma del teléfono.
Precisión	Los límites (monto ≤ 1 000 000 000, propina ≤ 100 %) existen porque `double` tiene ~15 cifras exactas.
Sin memoria	Al cerrar la app se pierde lo escrito; no hay historial (fuera del alcance del MVP).
Corredor de pruebas	Las pruebas del domain no usan widgets, pero corren con `flutter_test` porque el proyecto no admite `package:test`.

A12 · Cómo extenderlo

Un modo nuevo: "al múltiplo de 5"

```
// 1) lib/data/redondeo_multiplo_cinco.dart  (archivo NUEVO)
import 'package:divisor_cuenta/domain/estrategia_redondeo.dart';

/// Redondea hacia arriba al múltiplo de 5 más cercano: 27.5 → 30.
class RedondeoMultiploCinco implements EstrategiaRedondeo {
  @override
  double redondear(double montoPorPersona) =>
      ((montoPorPersona - 1e-9) / 5).ceilToDouble() * 5;
}

// 2) lib/main.dart: el import y una entrada en el mapa de crearControlador()
import 'package:divisor_cuenta/data/redondeo_multiplo_cinco.dart';
    'Múltiplo de 5': RedondeoMultiploCinco(),
```

Y nada más: la pantalla dibuja un botón por cada entrada del mapa. Se comprobó en una copia aparte del proyecto: con esos dos cambios, 34/34 pruebas en verde y el modo nuevo calcula 30.00 para 100 · 4 · 10 %. Agrega una prueba en `casos_borde_test.dart` y, si es un requisito nuevo, actualiza primero la spec.

Si quieres…	Toca solo
mostrar "$27.50"	`FormateadorMoneda.formatear`
cambiar un mensaje de error	las constantes de `ValidarEntrada` (y la spec, y las pruebas que lo buscan)
subir el monto máximo	`ValidarEntrada.montoMaximo` (revisa antes la precisión)
guardar un historial	una interfaz nueva en `domain/`, su implementación en `data/` y su registro en `main.dart`

A13 · La otra rama: qué hizo vibe con la misma app

Métrica	vibe	sdd
Escenarios de aceptación (0-6)	2	6
Iteraciones (correcciones)	2	0
Archivos / líneas en lib/	4 / 287	11 / 523
Pruebas propias que pasan	12	33
Pruebas de sdd que compilan	0	todas
Capas presentation/domain/data	no	sí
Modo nuevo sin tocar el cálculo	no	sí



vibe: 10 / 3 → $3.34 (se esperaba 3.33)

Primera respuesta de vibe (sin correcciones): 12 archivos, 1499 líneas, modo "por consumo" con platos, nombres de personas, botón "Copiar resumen", tema oscuro y el nombre de la app cambiado en Android, iOS y web. Nada de eso se pidió.

Versión final de vibe: 3 campos y cálculo en vivo. Falla los escenarios 3 a 6 porque decidió sola cosas que la frase no decía: redondear al centavo hacia arriba ($3.34), bloquear letras en el monto y usar otros textos de error.

> **La lección.** Las dos ramas usaron el mismo modelo. La diferencia no fue la inteligencia del agente: fue cuánta información recibió antes de empezar. Lo que no se escribe, el agente lo decide solo y en silencio; con SDD esas decisiones quedan en la spec, donde se pueden revisar y probar.

Parte B · Explicado como si tuvieras 8 años

Imagina que vas a comer pizza con tus amigos. Al final llega la cuenta y todos preguntan: **"¿cuánto pongo yo?"**. Esta app es una **calculadora mágica** que responde eso. Por dentro funciona como un **restaurante pequeñito** donde cada personaje tiene **un solo trabajo**.

Conoce al equipo



La gerente
`main.dart`
El primer día contrata a todos y le dice a cada uno con quién trabajar.



El mesero
`PantallaDivisor`
Te muestra el menú (los campos) y anota lo que escribes.



La jefa de meseros
`DivisorController`
Recibe la nota del mesero y organiza a todo el equipo.



El guardia
`ValidarEntrada`
Revisa que los números tengan sentido. Si no, dice ¡ALTO!



El chef
`CalcularDivision`
Hace la cuenta. Nunca habla con los clientes.



Gorro Exacto
`RedondeoExacto`
Cuando el chef se lo pone, corta en centavos: 3.33.



Gorro Arriba
`RedondeoHaciaArriba`
Con este gorro, todo sube al siguiente entero: 4.00.



El decorador
`FormateadorMoneda`
Escribe el número bonito, con dos decimales.

> **¿Por qué cada uno hace una sola cosa?** Si el chef también tuviera que cuidar la puerta y decorar los platos, se equivocaría más y nadie sabría a quién regañar cuando algo sale mal. Con un trabajo por personaje, si el número sale feo ya sabes que es culpa del decorador. Los programadores le dicen a esto **SRP** (una sola responsabilidad).

El viaje de un número (cómic)

Tú y tres amigos (4 personas) comieron por **100 dólares** y quieren dejar **10 %** de propina.

1



Escribiste **100**, **4** y **10**. ¡Tocaste Calcular! Se lo llevo a la jefa.

2



Son letras en un papel. Los convierto en números de verdad: 100, 4 y 10.

3



¿Cien es más que cero? Sí. ¿Hay al menos una persona? Sí. ¡Pasen!

4



100 más la propina son 110. Entre 4 amigos… ¡27.5 cada uno!

5



Soy el gorro Exacto. 27.5 ya está en centavos justos, lo dejo así.

6



Le pongo dos decimalitos para que se vea bonito:

27.50

¿Y si alguien escribe "abc"? El guardia mira el papel, ve que "abc" no es un número y grita **"¡Monto inválido!"**. La jefa le pasa el mensaje al mesero, que lo muestra en rojo. **El chef ni se entera:** nunca cocina con ingredientes malos.



¿Y el otro gorro? Si en la pantalla eliges "Hacia arriba", la jefa le da al chef el gorro Arriba. Con 10 dólares entre 3 amigos salen 3.33…, y con ese gorro se vuelve **4.00**: todos ponen un poquito más y nadie se queda sin pagar. Lo bonito es que el chef hace exactamente lo mismo con cualquier gorro: no le importa cuál le pongan.

La casa de tres pisos (las capas)



El código está ordenado como un **pastel de tres pisos**.

Arriba está lo que ves: los cuadritos para escribir y el botón. Los hizo Flutter, que es como una caja de LEGO para hacer pantallas.

En el medio están las **reglas**: cómo se divide y cuándo un número está mal. Este piso es especial: **no sabe que existe Flutter**. Podrías cambiar toda la pantalla por otra y las reglas seguirían iguales.

Abajo están las **herramientas**: los gorros de redondear.

Y la **vela** (main.dart) es lo que enciende todo: junta los pisos el primer día.

Dos maneras de cocinar (vibe vs SDD)



Vibe: "¡hazme algo rico!"
Le dices al cocinero una sola frase y él adivina el resto. Te trajo una pizza con **diez ingredientes que no pediste**. Le pediste que la hiciera más simple dos veces, y aun así le puso la salsa a su manera: cuando 10 dólares se dividen entre 3, cobra 3.34 en vez de 3.33.



SDD: primero la receta
Antes de cocinar se escriben las **reglas de la cocina** (constitución), la **receta** (spec), se hacen **preguntas** ("¿y si alguien pone 0?"), se arma la **lista de pasos** (tareas) y alguien **revisa** que todo cuadre. Después se cocina. Salió exactamente lo pedido: **6 de 6**.



¿Y las pruebas? Son **33 exámenes sorpresa** que la app tiene que aprobar cada vez que alguien cambia algo. Si un día alguien rompe el gorro Exacto, el examen número 5 sale en rojo y lo sabemos al instante, antes de que un amigo pague mal.

Glosario

Cada término tiene su definición técnica y, en rosado, la versión para niños.

SDD (Spec-Driven Development)
escribir qué debe hacer el programa antes de programarlo. Como escribir la receta antes de cocinar.

Vibe coding
pedirle código al agente sobre la marcha, sin especificación previa. Cocinar "a ojo".

Spec Kit
herramienta de GitHub que guía al agente por constitución, spec, plan, tareas e implementación. El cuaderno de recetas con casillas para llenar.

Constitución
reglas permanentes del proyecto que todo debe cumplir. Las reglas de la cocina: "nunca cocinar sin lavarse las manos".

Spec
qué debe hacer la app y cómo se sabe que lo hace (escenarios de aceptación). La receta.

Clarify
paso donde el agente pregunta lo ambiguo antes de planificar. "¿La pizza con o sin cebolla?"

Plan / tareas
cómo se va a construir y la lista numerada de pasos (T001…). La lista de pasos de la receta.

Analyze / converge
revisiones automáticas antes y después de programar. El inspector de la cocina.

SOLID
cinco principios de diseño: SRP, OCP, LSP, ISP, DIP. Cinco reglas para que los personajes no se pisen.

SRP
una clase, una razón de cambio. Un trabajo por personaje.

OCP
agregar sin modificar lo que ya funciona. Un gorro nuevo sin cambiar al chef.

LSP
cualquier implementación sirve donde se espera la interfaz. El chef cocina igual con cualquier gorro.

ISP
interfaces pequeñas. El gorro solo sabe hacer una cosa.

DIP
depender de abstracciones, no de detalles. La jefa pide "un gorro", no "el gorro rojo".

Capa
grupo de archivos con un mismo tipo de trabajo. Un piso del pastel.

Interfaz
contrato que dice qué métodos existen, sin decir cómo funcionan. La forma del gorro: cualquiera que tenga esa forma sirve.

Raíz de composición
el único lugar donde se crean y conectan las piezas (main.dart). La gerente el primer día.

Widget
pieza de pantalla en Flutter. Un bloque de LEGO.

setState
le avisa a Flutter que redibuje la pantalla. "¡Mesero, actualiza la pizarra!"

Prueba unitaria / de widget
código que comprueba una función o una pantalla automáticamente. Un examen sorpresa.

flutter analyze / lint
revisión automática del estilo y de errores comunes. El corrector de ortografía del código.

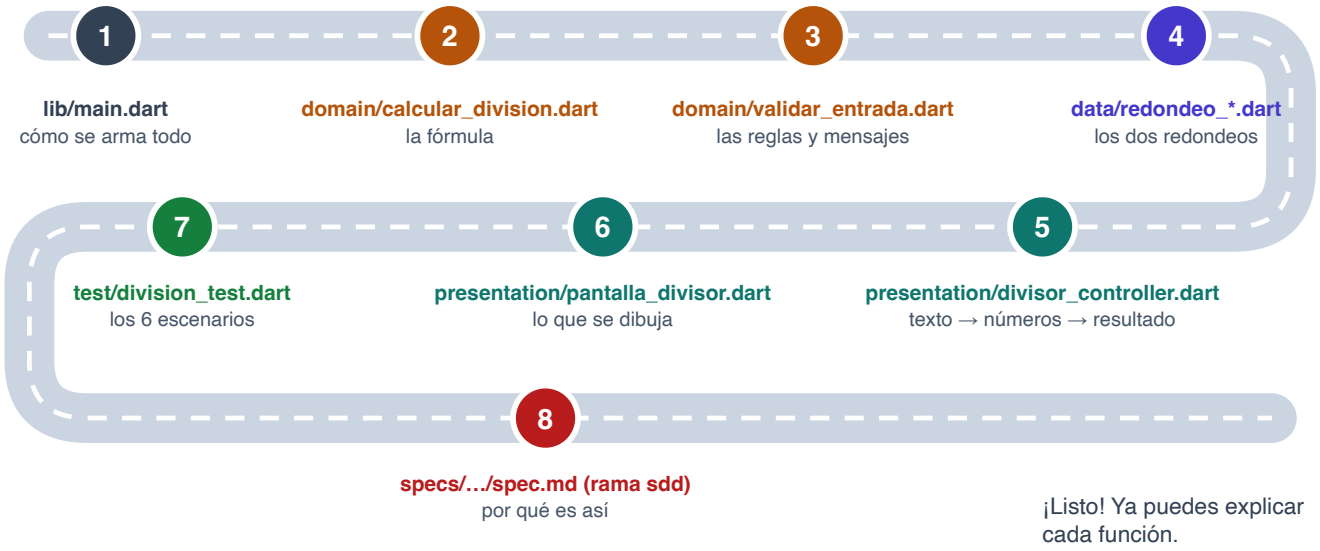
Punto flotante
forma en que la computadora guarda decimales; algunos (como 1.1) no caben exactos. Una regla que no tiene la marca exacta de 1.1.

Commit / rama
foto guardada del proyecto / línea de historia paralela. Una foto del álbum / un álbum aparte.

MVP
la versión más pequeña que ya resuelve el problema. Una pizza simple pero rica.

Dónde mirar primero en el código

Lee en este orden (unos 20 minutos). Cada parada responde una pregunta; al llegar al final puedes explicar cualquier función.



#	Archivo	Pregunta que responde
1	`lib/main.dart`	¿Qué piezas existen y quién las conecta?
2	`lib/domain/calcular_division.dart`	¿Cuál es la fórmula? (3 líneas de lógica)
3	`lib/domain/validar_entrada.dart`	¿Qué entradas se rechazan y con qué mensaje?
4	`lib/data/redondeo_exacto.dart`, `redondeo_hacia_arriba.dart`	¿Cómo se redondea y por qué la tolerancia?
5	`lib/presentation/divisor_controller.dart`	¿Cómo se pasa del texto escrito al resultado?
6	`lib/presentation/pantalla_divisor.dart`	¿Qué se dibuja y cuándo se redibuja?
7	`test/casos_de_prueba.dart` + `test/division_test.dart`	¿Cómo se demuestra que cumple la spec?
8	`specs/001-divisor-cuenta/spec.md`, `.specify/memory/constitution.md`	¿Por qué el código es así y no de otra forma?

> Truco para la defensa oral: abre `calcular_division.dart` y explica por qué **no** tiene `if`, ni mensajes, ni `toStringAsFixed`. Si puedes explicar esas tres ausencias, entendiste SRP, OCP y LSP.